Homework 7: Async Order Processing (Part II and Part III)

Course: CS 6650 Building Scalable Distributed Systems, Northeastern University

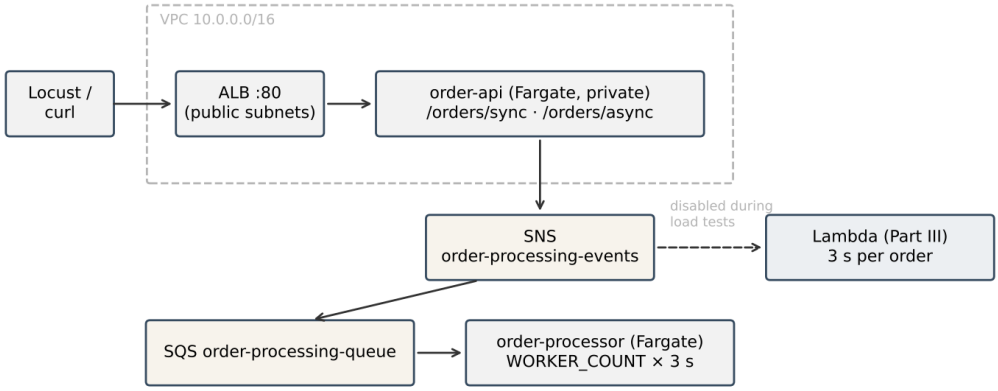
Date: November 8, 2026

Team member	Repository
Santosh Vasisht Sunkara	https://github.com/santosh50/CS6650_2b_demo/tree/master/HW07
Sameep Anil Sawant	https://github.com/samsaw777/BSDS_Course_Work/tree/main/HW07
Nikhil Shivakumar	github.com/nikhilsk98/distributed-systems-course/hw07

1. Overview and setup

- **Problem:** each order needs a 3 s payment verification and the payment processor handles one order at a time. Under flash sale load, a synchronous API makes customers wait in line, and throughput caps at about 0.33 orders/s.
- **Implemented (all three):** a synchronous order API (payment inside the request), an asynchronous order API (Receiver → SNS → SQS → Processor workers), and an SNS-triggered Lambda processor for comparison.
- **Deployment (all three):** Go services on AWS ECS Fargate in us-east-1 (Learner Lab, LabRole), provisioned with Terraform. VPC 10.0.0.0/16, public subnets 10.0.1.0/24 and 10.0.2.0/24 for the ALB, private subnets 10.0.10.0/24 and 10.0.11.0/24 for ECS, NAT gateway for outbound access. Tasks use 256 CPU units and 512 MB, with a /health endpoint.

Setting	Santosh	Sameep	Nikhil
Payment gate	Buffered channel, capacity 1	Buffered channel, capacity 1	Buffered channel, capacity 1
Locust	Web UI, wait 0.1–0.5 s, no client timeout, laptop in Boston	wait 0.1–0.5 s, no client timeout, Boston	Locust 2.47 headless, wait 0.1–0.5 s, laptop
Worker count control	terraform apply -var worker_count=N	terraform apply -var processor_workers=N	terraform apply -target=…processor -var worker_count=N, min healthy 0%
Processor log	Every 10 s: workers, processed, in_flight, rate_10s, avg_rate	Every 10 s: workers, processed, in_flight, avg_rate	Every 15 s: processed, active, workers
Queue depth source	AWS CLI ApproximateNumberOfMessages (visible) every ~11 s, plus CloudWatch	CloudWatch ApproximateNumberOfMessagesVisible	Visible + in flight sampled every 5 s, plus CloudWatch
CloudWatch statistic	Maximum, 1 min	Average, 1 min	Maximum, 1 min
Processor poll design	Requests only as many messages as free workers (≤ 10)	Fixed pool of N workers	Requests only as many messages as free workers (≤ 10)
Extra test	—	—	Async normal test (5 users, 30 s)



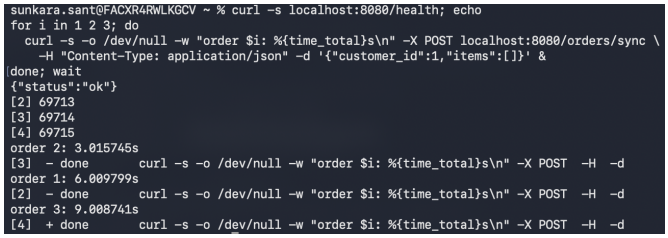
Nikhil · Fig 0. Architecture used by all three of us. The Lambda subscription was enabled only after every Part II load test had finished.

2. Phase 1: Synchronous system

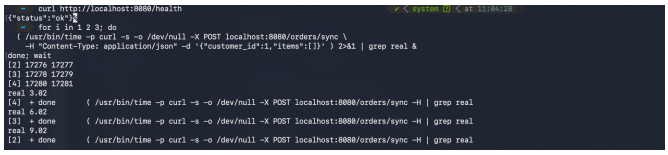
Test setup (all three): Locust posting to POST /orders/sync through the ALB. Normal test: 5 users, spawn rate 1, 30 s. Flash sale test: 20 users, spawn rate 10, 60 s.

Gate check: Santosh (3 concurrent local requests): 3.015745 s, 6.009799 s, 9.008741 s. Sameep (3 concurrent local requests): 3.02 s, 6.02 s, 9.02 s. Nikhil (deployed smoke test): /orders/sync → 200 in 3.06 s, /orders/async → 202 in 0.12 s.

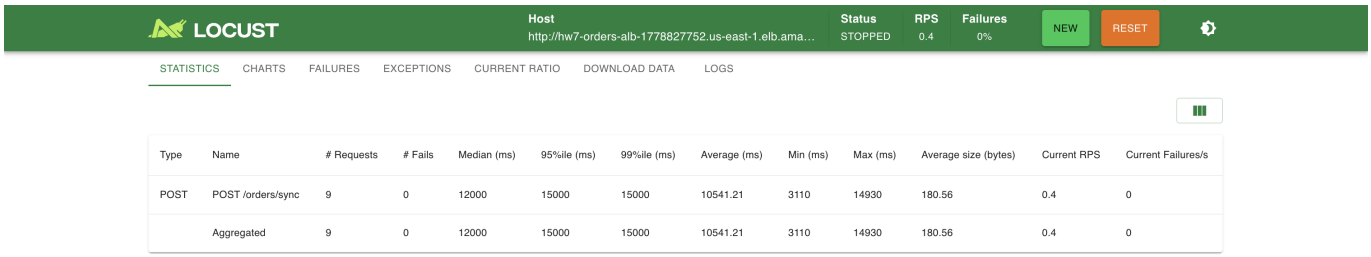
Run	Test	Requests	Failures	Median	p95	Average	Min / Max	Throughput
Santosh	Normal	9	0	12000 ms	15000 ms	10541.21 ms	3110 / 14930 ms	0.3/s
Sameep	Normal	9	0	12000 ms	15000 ms	—	— / 14837 ms	~0.33/s
Nikhil	Normal	9	0	12000 ms	15000 ms	10451.31 ms	3079 / 14748 ms	0.33/s
Santosh	Flash	19	0	30000 ms	56000 ms	29665.12 ms	3106 / 56164 ms	0.3167/s
Sameep	Flash	19	0	30000 ms	56000 ms	—	— / 56163 ms	~0.32/s
Nikhil	Flash	19	0	30000 ms	56000 ms	29620.7 ms	3082 / 56086 ms	0.33/s



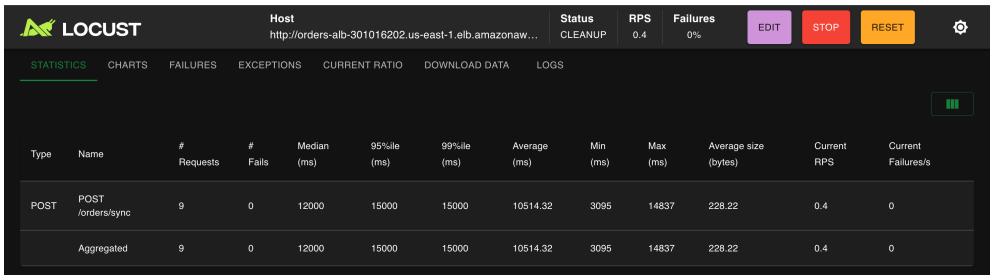
Santosh · Fig 1a. Local gate check: 3.02 / 6.01 / 9.01 s



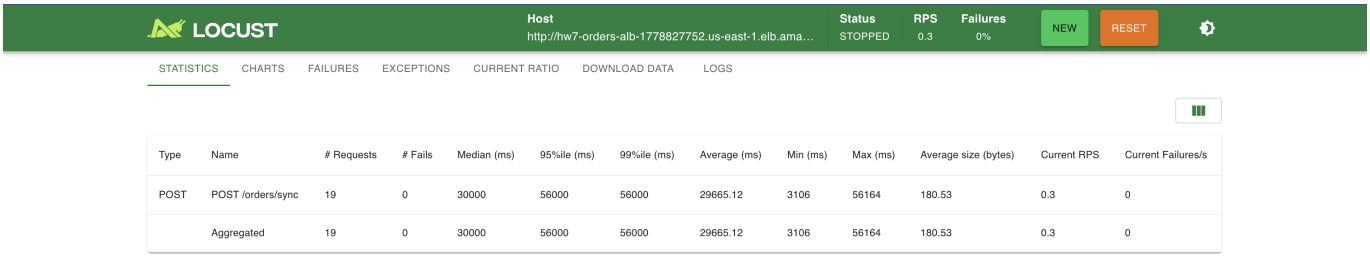
Sameep · Fig 1b. Local gate check: 3.02 / 6.02 / 9.02 s



Santosh · Fig 2a. Sync normal: 9 requests, median 12000 ms, max 14930 ms



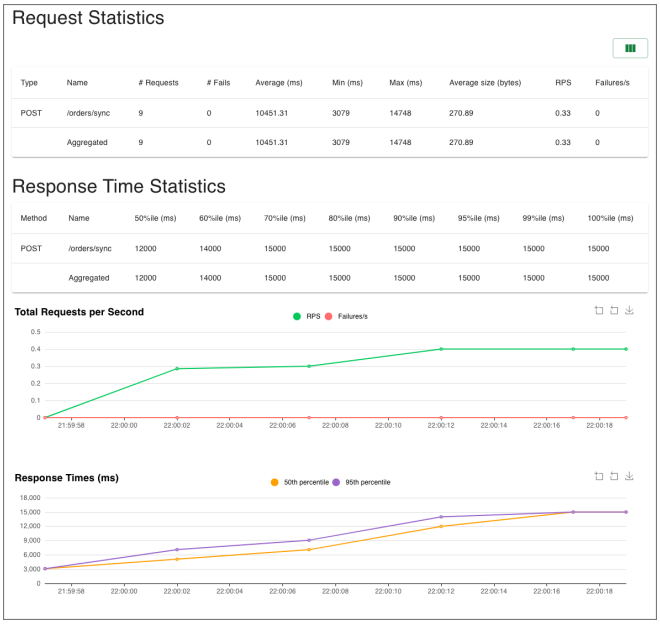
Sameep · Fig 2b. Sync normal: 9 requests, median 12000 ms, max 14837 ms



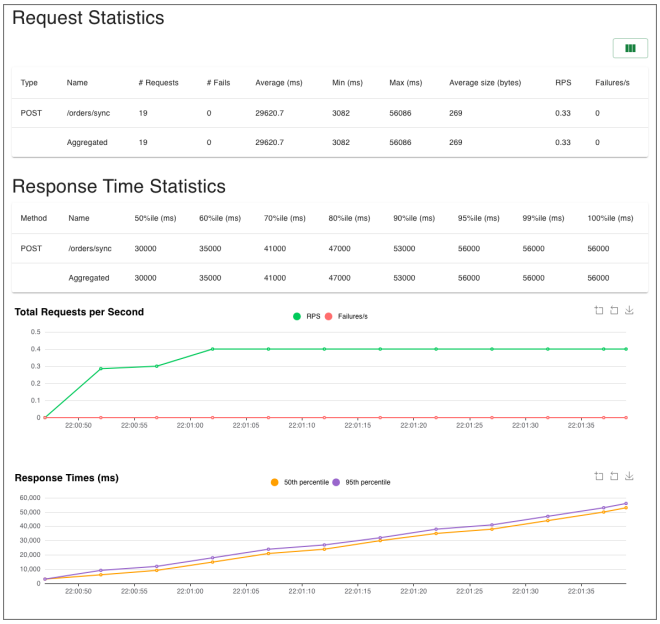
Santosh · Fig 3a. Sync flash: 19 requests, median 30000 ms, max 56164 ms



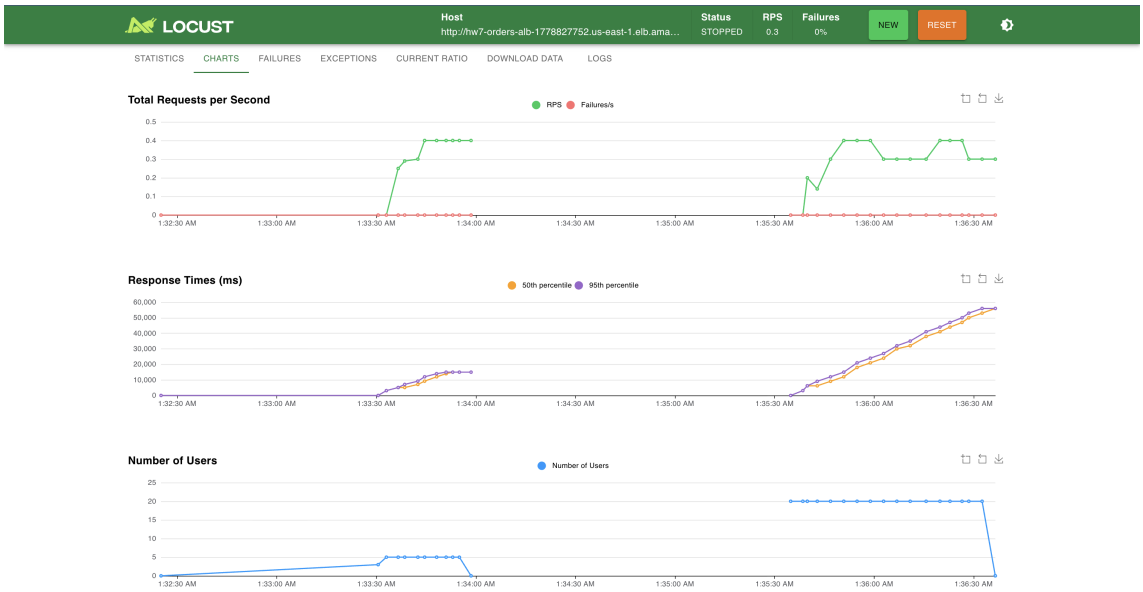
Sameep · Fig 3b. Sync flash: 19 requests, median 30000 ms, max 56163 ms



Nikhil · Fig 2c. Sync normal: 9 requests, 0 failures, 0.33 RPS, max 14748 ms



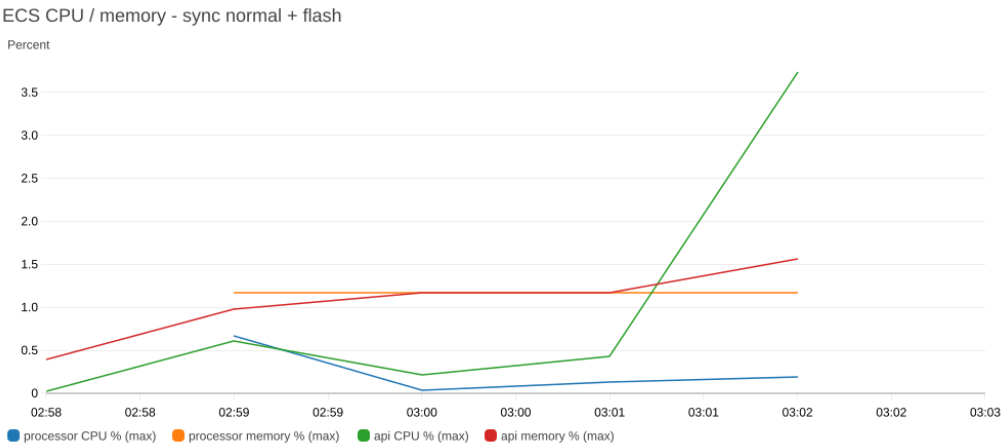
Nikhil · Fig 3c. Sync flash: 19 requests, 0 failures, median 30000 ms, max 56086 ms



Santosh · Fig 4a. Locust charts for both sync runs: RPS flat at 0.3–0.4 while response time climbs to ~15 s, then ~56 s



Sameep · Fig 4b. Locust charts for both sync runs



Nikhil · Fig 4c. API and processor CPU/memory during the sync tests: the API stays under 4% CPU, so compute is not the bottleneck

Observations (all three):

- Throughput stayed at ~0.33/s with 4× more users. Extra users only made the line longer.
- Max wait grows with users × 3 s: ~15 s at 5 users, ~56 s at 20 users (the n-th customer waits about n × 3 s).
- 0% failures is misleading. With no client timeout, customers waited up to 56 s instead of failing, right at the ALB's 60 s idle timeout; a slightly longer test would return HTTP 504s.
- Requests still waiting when Locust stopped were never counted, so 19 understates the customers affected.
- Nikhil measured API CPU under 4% during the sync tests: the bottleneck is the payment step, not compute.

Comparison: all three sync runs are practically identical: same request counts, same median and p95, and max values within 78 ms of each other (56086–56164 ms). The single-slot gate behaves the same in every implementation, as expected for a 3 s serial bottleneck that no network or AWS setting can change.

3. Phase 2: Bottleneck analysis

Question	Theoretical	Santosh	Sameep	Nikhil
Max throughput with 20 customers	1 / 3 s = 0.33 orders/s	19 / 60 s = 0.3167/s	19 / 60 s = ~0.32/s	0.33 req/s (19 orders in 57 s)
Orders lost per second at 20/s	20 − 0.33 ≈ 19.67/s	0 failures; waited up to 56164 ms	Users wait in line instead of being dropped	Rest of the demand never served

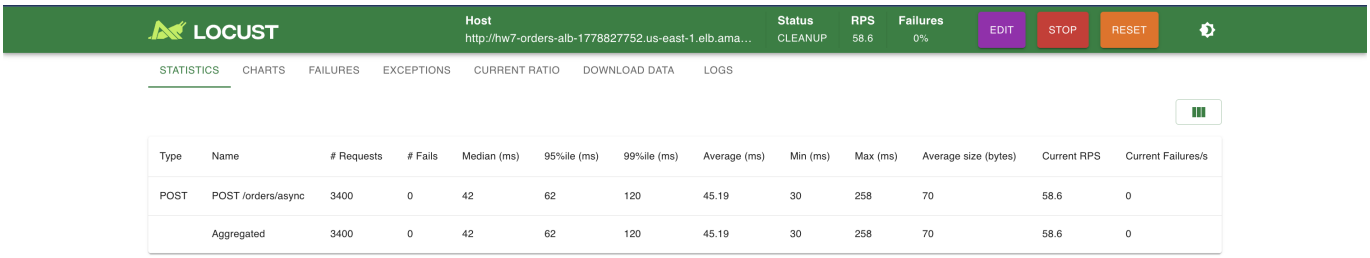
- The measured rate matches theory for all three of us. One payment slot caps throughput no matter how many customers arrive.
- **Where the ~57/s async demand comes from (Nikhil):** each Locust user sends one request per (response time + mean wait). With ~45 ms responses and a 0.3 s mean wait, 20 users generate 20 ÷ 0.345 ≈ 58 orders/s, which is the handout's ~60/s scenario. All three of us measured 56–57/s.
- **What can change:** payment time cannot be reduced, but the customer does not need to wait for it. Accept the order immediately and verify payment later. Adding API tasks would not help, because the payment provider is the limit.

4. Phase 3: Async solution

Test setup (all three): same flash profile (20 users, spawn rate 10, 60 s), Locust posting to POST /orders/async, processor running 1 worker.

Run	Accepted	Failures	Median	p95	p99	Max	Accept rate	vs sync
Santosh	3400	0	42 ms	62 ms	120 ms	258 ms	56.67/s	178.95× (3400 ÷ 19)
Sameep	3343	0	47 ms	110 ms	140 ms	272 ms	~56/s	~176× (3343 ÷ 19)
Nikhil	3414	0	43 ms	56 ms	100 ms	276 ms	57.1/s	~180× (3414 ÷ 19)

- All three of us got 0 failures and median acceptance of 42–47 ms.
- Santosh: p95 spiked to ~110 ms only while users ramped up, then settled at 55–70 ms. The Locust UI stayed in CLEANUP after the run; the stats did not change for 3 minutes, so they are final.
- Sameep: p95 110 ms is slightly above the 100 ms target, attributed to network latency plus the SNS publish call.
- Nikhil also ran an async normal test (5 users, 30 s): 400 requests, 0 failures, 13.4 req/s, median 44 ms, p95 61 ms.



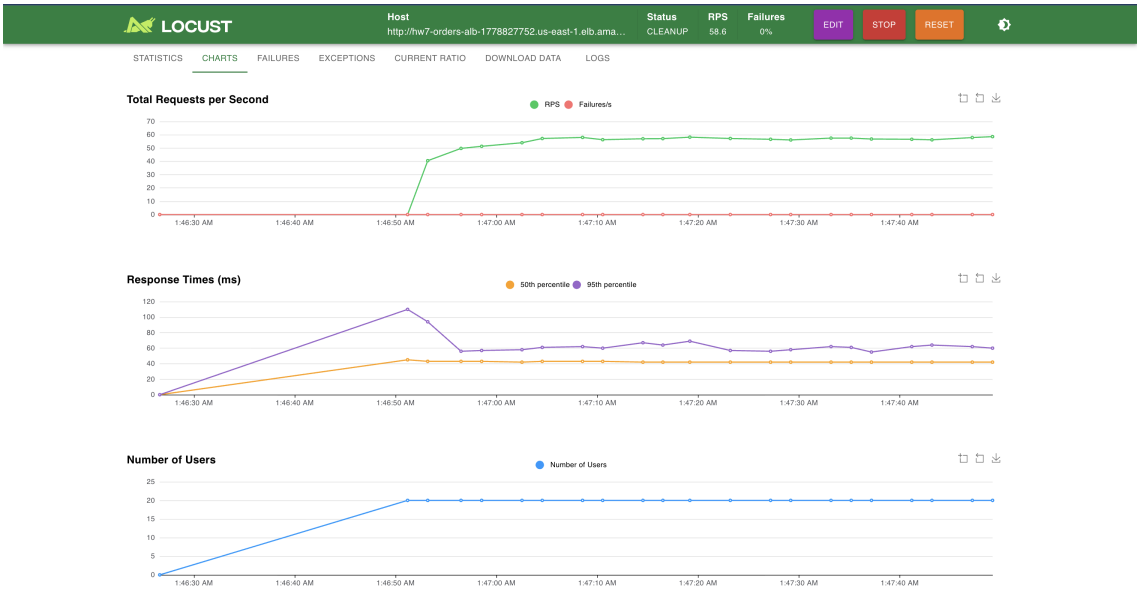
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	POST /orders/async	3400	0	42	62	120	45.19	30	258	70	58.6	0
	Aggregated	3400	0	42	62	120	45.19	30	258	70	58.6	0

Santosh · Fig 5a. Async flash, 1 worker: 3400 requests, 0 failures, median 42 ms, p95 62 ms

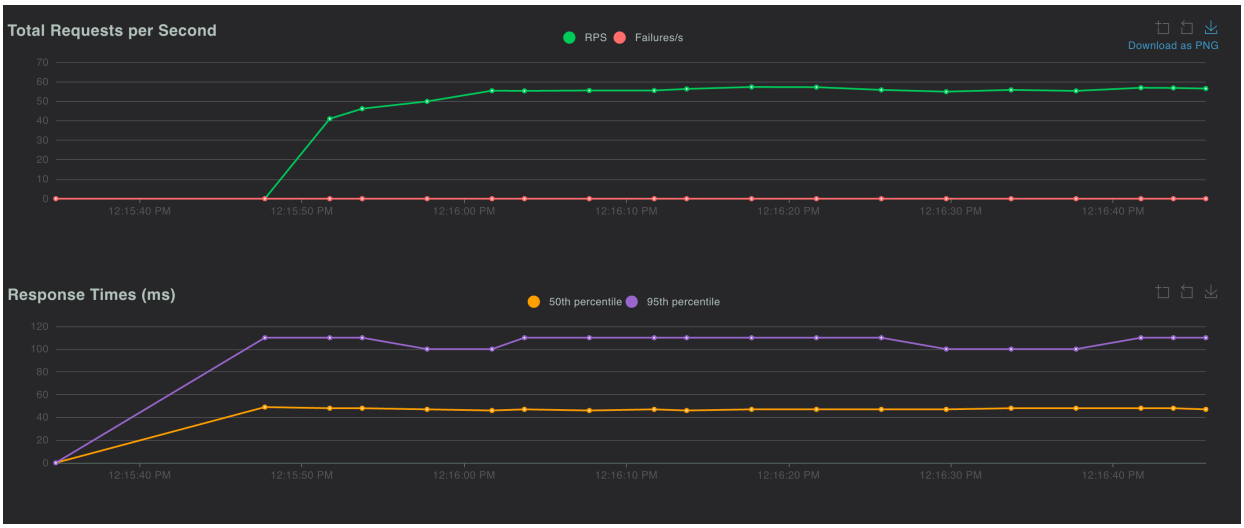


Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	POST /orders/async	3343	0	47	110	140	56.57	31	273	243.94	56.5	0
	Aggregated	3343	0	47	110	140	56.57	31	273	243.94	56.5	0

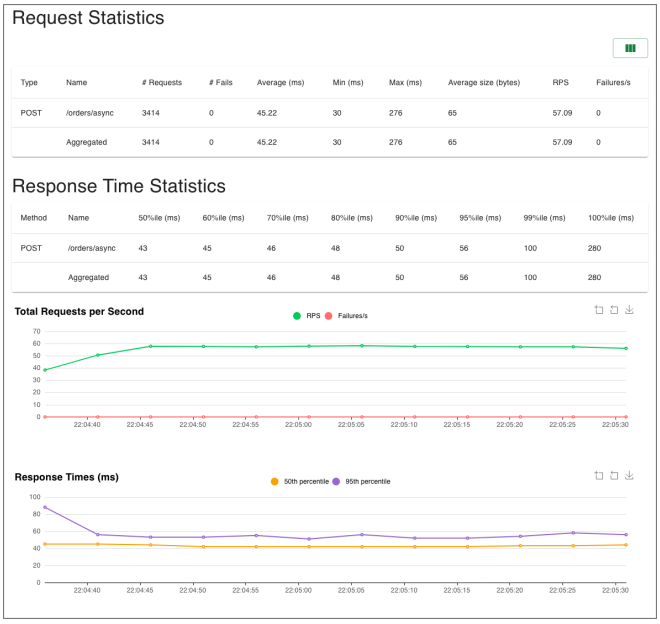
Sameep · Fig 5b. Async flash, 1 worker: 3343 requests, 0 failures, median 47 ms, p95 110 ms



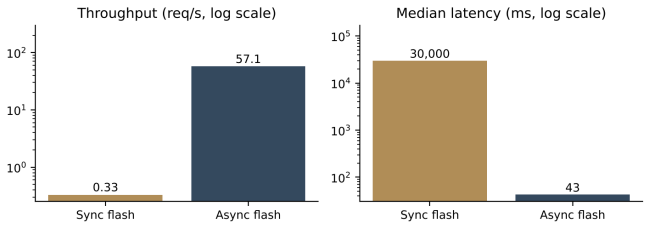
Santosh · Fig 6a. Async charts: ~55–58 RPS, median ~42 ms



Sameep · Fig 6b. Async charts: ~56 RPS, median ~47 ms, p95 ~110 ms



Nikhil · Fig 5c/6c. Async flash, 1 worker: 3414 requests, 0 failures, 57.09 RPS, median 43 ms, p95 56 ms



Nikhil · Fig 6d. Sync vs async flash sale on log scales: 0.33 vs 57.1 req/s, 30000 vs 43 ms median

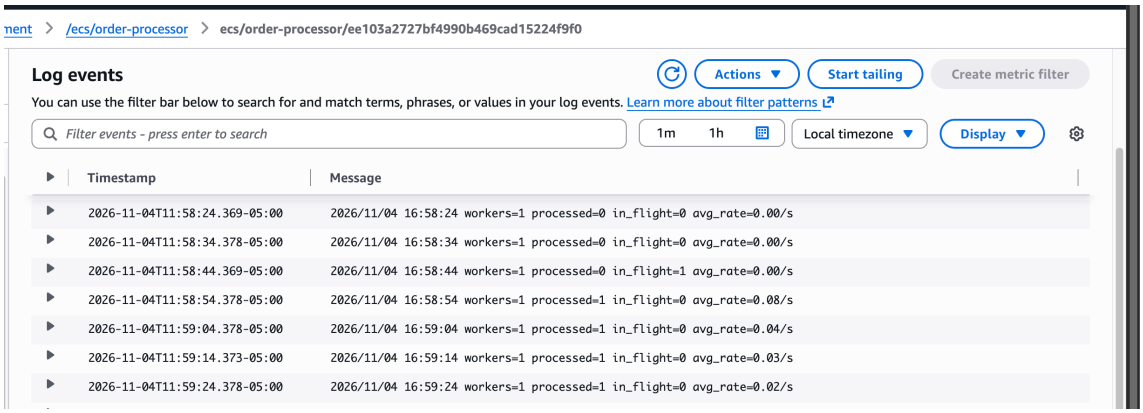
Accepted is not processed

Stage	Description	Santosh	Sameep	Nikhil
1. Accepted	Receiver publishes to SNS, returns HTTP 202	3400, 0 failures	3343, 0 failures	3414, 0 failures
2. Processed	Processor pulls from SQS, runs the 3 s payment	20 by end of test	~20 by end of test	≈ 20 by end of test (5 per 15 s)

- Locust only measures stage 1: “3400 requests, 0 failures” means 3400 orders accepted, not 3400 paid.
- In the sync design both stages happen in the same request, so the Locust count (19) was also the processed count.
- Measuring the async system needs queue metrics and the processor log.

```
2026-11-09T06:28:00.044000+00:00 ecs/processor/36764693d4204fe1bc9fcb17caf08007 2026/11/09 06:28:00 workers=1 processed=0 in_flight=0 rate_10s=0.00/s avg_rate=0.00/s
2026-11-09T06:28:10.044000+00:00 ecs/processor/36764693d4204fe1bc9fcb17caf08007 2026/11/09 06:28:10 workers=1 processed=0 in_flight=0 rate_10s=0.00/s avg_rate=0.00/s
2026-11-09T06:28:20.044000+00:00 ecs/processor/36764693d4204fe1bc9fcb17caf08007 2026/11/09 06:28:20 workers=1 processed=0 in_flight=0 rate_10s=0.00/s avg_rate=0.00/s
2026-11-09T06:28:30.039000+00:00 ecs/processor/36764693d4204fe1bc9fcb17caf08007 2026/11/09 06:28:30 workers=1 processed=1 in_flight=0 rate_10s=0.10/s avg_rate=0.11/s
2026-11-09T06:28:40.044000+00:00 ecs/processor/36764693d4204fe1bc9fcb17caf08007 2026/11/09 06:28:40 workers=1 processed=1 in_flight=0 rate_10s=0.00/s avg_rate=0.05/s
```

Santosh · Fig 7a. Smoke test: processed goes from 0 to 1 at 06:28:30 UTC



Sameep · Fig 7b. First test order: processed goes from 0 to 1

```
2026/11/08 03:06:21 stats processed=72 active=1 workers=1
2026/11/08 03:06:36 stats processed=77 active=1 workers=1
2026/11/08 03:06:51 stats processed=82 active=1 workers=1
2026/11/08 03:07:06 stats processed=87 active=1 workers=1
2026/11/08 03:07:21 stats processed=92 active=1 workers=1
2026/11/08 03:07:36 stats processed=97 active=1 workers=1
```

Nikhil · Fig 7c. Processor log with 1 worker: exactly 5 orders per 15 s while the API accepted ~57 per second

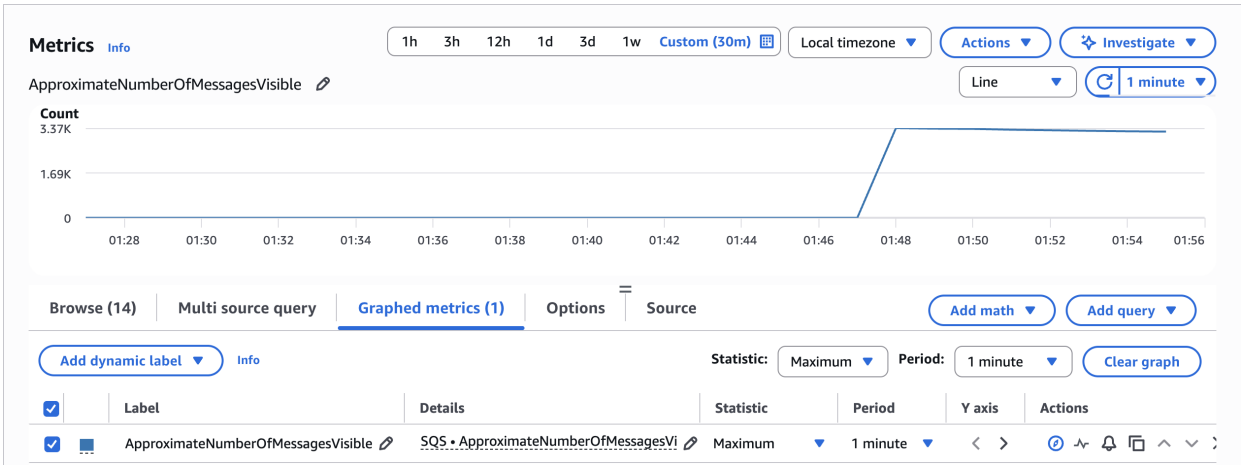
Comparison: acceptance is the same for all three (56–57/s, 0 failures, ~176–180× more than sync, 42–47 ms median). The p95 values split 56 / 62 / 110 ms. Server-side work is identical (one SNS publish), so the gap comes from the client side: the network path and time of day for each laptop running Locust.

5. Phase 4: Queue buildup (1 worker)

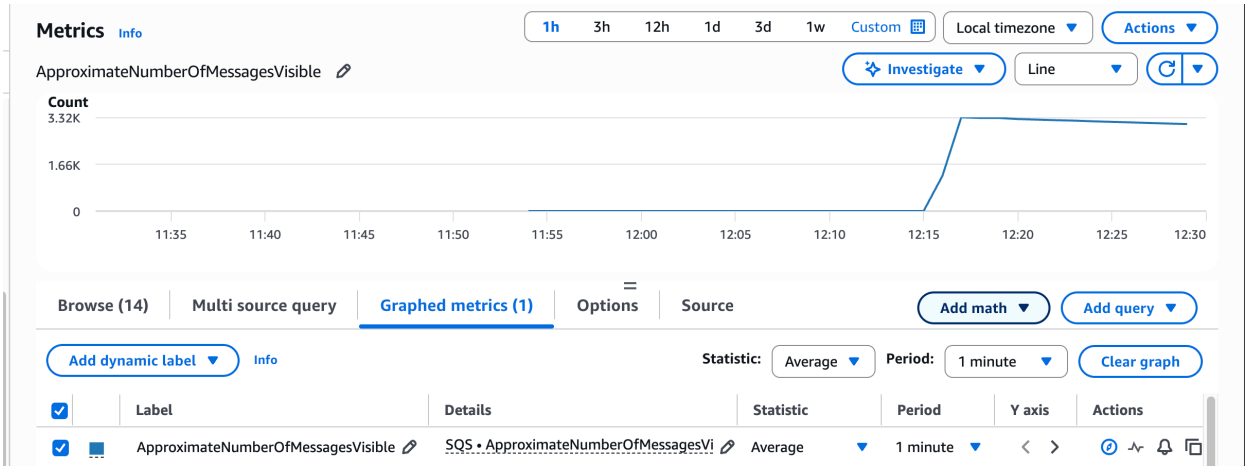
Item	Santosh	Sameep	Nikhil
Peak queue depth	3378 (CLI, visible) at 01:48:08 local	~3,320 (CloudWatch)	3,396 (5 s samples, visible + in flight); 3,384 on CloudWatch
Measured processing rate	0.3303/s (109 in 330 s); 0.3348/s over 11 min before purge	~0.35/s (~340 in 16 min)	0.333/s (5 per 15 s; ~20/min deleted)
Order acceptance rate	56.67/s	~56/s	57.1/s
Queue growth rate	56.67 − 0.3303 = 56.34/s	56 − 0.33 = ~55.7/s	57.1 − 0.33 = 56.8/s
Time to clear actual backlog	3378 ÷ 0.3303 = 10,227 s ≈ 170.5 min	3,320 ÷ 0.33 ≈ 167 min	3,396 × 3 s = 10,188 s ≈ 170 min
Full 1-hour flash sale at 60/s	≈ 214,800 msgs ÷ 0.33 ≈ 10,740 min (~7.5 days)	≈ 10,700 min (~7.5 days)	≈ 10,740 min (~7.5 days)
Time to zero	Not measured: purged at 01:59:08 local	Not measured: purged	Not measured: purged at 03:09:46 UTC (3,396 → 3,316 over the 4 min after the sale)
Processor CPU / memory	1.9163025618% / 1.3671875% (peak)	~0.5% / ~1.3–1.5%	0.34% / 1.4% (max)

- The worker was busy the whole time but far too slow: orders arrived ~170× faster than one worker could pay for them.

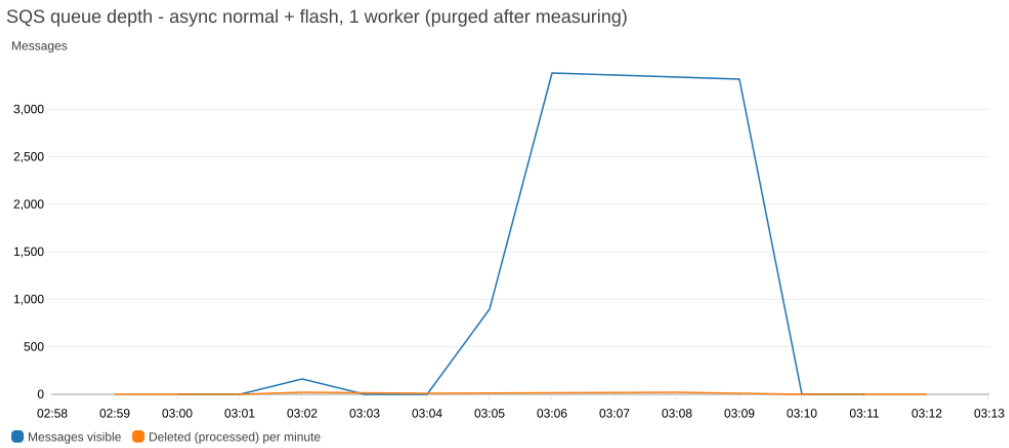
- No orders were lost. The queue stores work durably but does not add capacity; the backlog only clears when processing is faster than arrivals.
- The 1-hour case takes longer to clear than SQS's 4-day retention, so with one worker about half the orders would expire unprocessed.
- ApproximateNumberOfMessages is approximate: Santosh's CLI readings jumped around during load (761, 0, 2005, 1847…) because SQS spreads the queue across servers. Readings settle once load stops.
- **Customer impact:** the last flash-sale customer gets a confirmation almost 3 hours after clicking buy; for a full one-hour sale, a week or never.



Santosh · Fig 8a. Queue depth, 1 worker: jumps to ~3.37K at 01:48, then an almost flat decline



Sameep · Fig 8b. Queue depth, 1 worker: spike to ~3.32K, then an almost flat decline



Nikhil · Fig 8c. Queue depth, 1 worker: the small bump is the async normal test; the flash sale jumps to ~3,400 and barely moves (deleted ≈ 20/min) until the purge at 03:09:46

Comparison: peaks track accepted orders (3343 → 3400 → 3414 accepted gives ~3,320 → 3378 → 3,396). Nikhil's peak counts in-flight messages too, so it sits slightly higher. All three measured one worker at ≈ 0.33/s and projected ~2.8 hours to clear.

6. Phase 5: Scaling workers

Test setup (all three): async flash test (20 users, spawn rate 10, 60 s) repeated with 1, 5, 20 and 100 worker goroutines in the same 256 CPU / 512 MB task, with the queue at 0 and only the new task running before each run.

Request acceptance (Locust)

Workers	Santosh: accepted / fails / median / p95	Sameep: accepted / fails / median / p95	Nikhil: accepted / fails / RPS
1	3400 / 0 / 42 ms / 62 ms	3,343 / 0 / 47 ms / 110 ms	3,414 / 0 / 57.09
5	3448 / 0 / 43 ms / 60 ms	3,320 / 0 / 49 ms / 110 ms	3,420 / 0 / 57.15
20	3453 / 0 / 43 ms / 59 ms	3,318 / 0 / 48 ms / 110 ms	3,412 / 0 / 57.01
100	3431 / 0 / 45 ms / 61 ms	3,361 / 0 / 47 ms / 110 ms	3,414 / 0 / 57.04

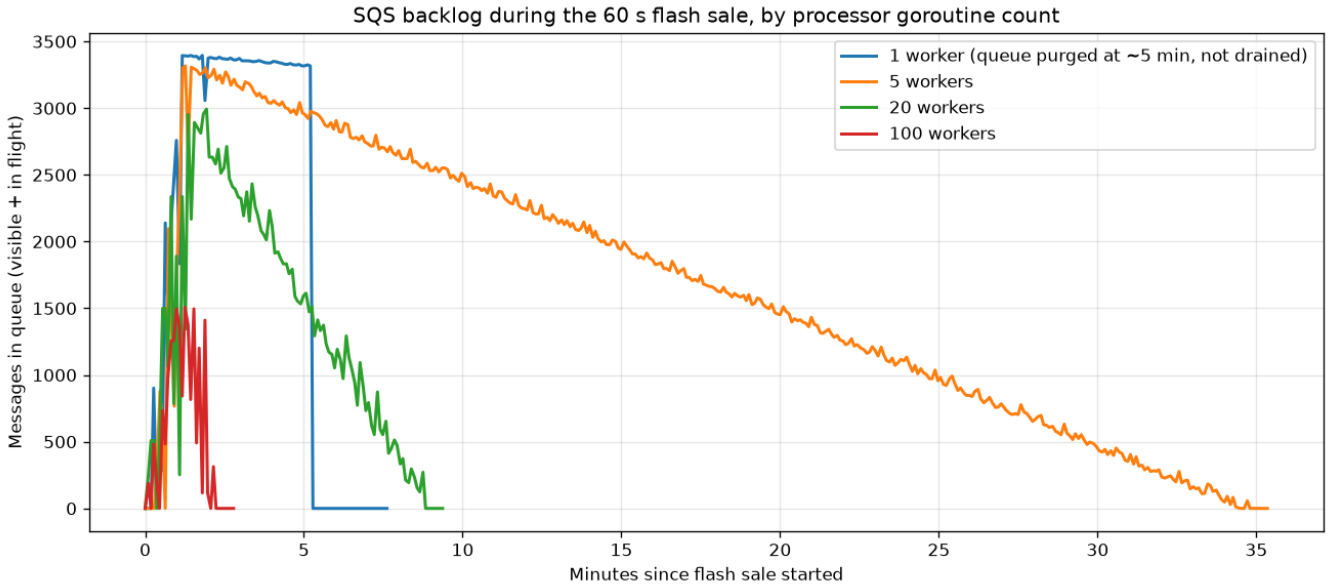
For all three of us, acceptance does not depend on worker count. Only the processing side changes.

Processing results

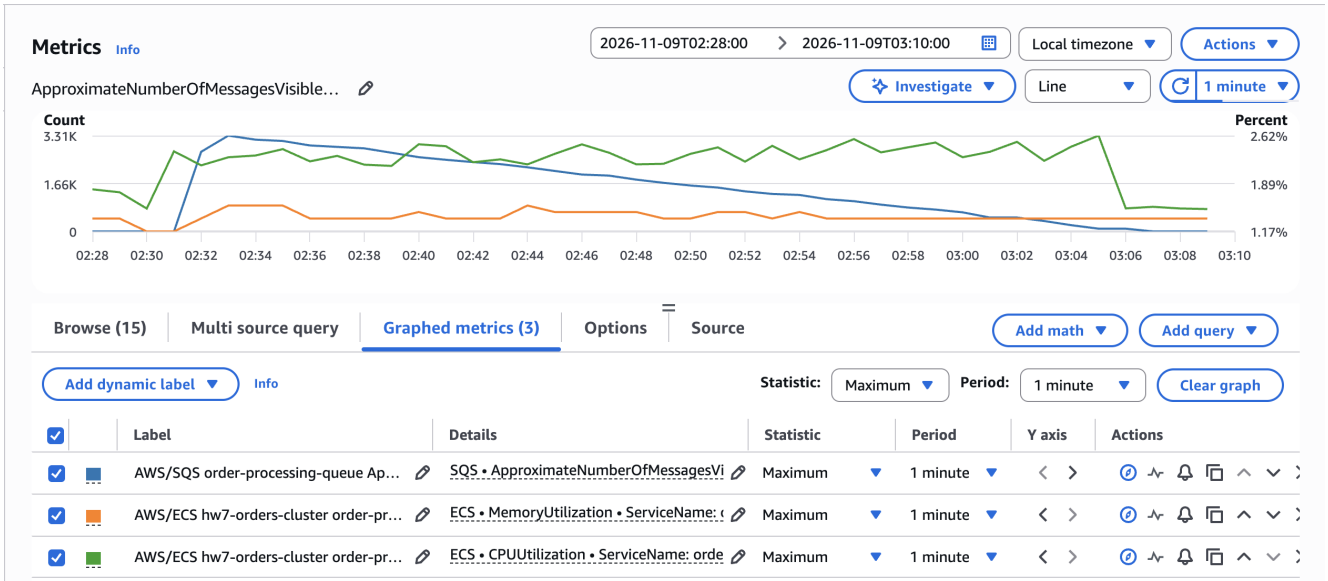
Workers	Run	Theory	Measured rate	Peak depth	Time to empty	Accepted / processed
1	Santosh	0.33/s	0.3303/s	3378	Purged (~170.5 min projected)	3400 / 247 before purge
1	Sameep	0.33/s	~0.35/s	~3,320	Purged	3,343 / —
1	Nikhil	0.33/s	0.333/s	3,396	Purged (~2 h 50 min calculated)	3,414 / —
5	Santosh	1.67/s	1.650/s	3345	34 min 52 s from sale start (33 min 52 s after load)	3448 / 3450
5	Sameep	1.67/s	~1.65/s	~3,057+	~31 min (projected)	3,320 / —
5	Nikhil	1.67/s	≈ 100/min (1.67/s)	3,317	34 min 32 s from sale start	3,420 / —
20	Santosh	6.67/s	6.588/s	2974	8 min 53 s from sale start (7 min 54 s after load)	3453 / 3454

Workers	Run	Theory	Measured rate	Peak depth	Time to empty	Accepted / processed
20	Sameep	6.67/s	~6.5/s	~2,900	Not measured	3,318 / —
20	Nikhil	6.67/s	≈ 400/min (6.67/s)	2,993	8 min 50 s from sale start	3,412 / —
100	Santosh	33.33/s	33.22/s	1352	~1 min 55 s from sale start (~1 min after load)	3431 / 3431
100	Sameep	33.3/s	~31.5/s	~1,250	~1–2 min after load	3,361 / —
100	Nikhil	33.3/s	33.3/s (500 per 15 s)	1,507	2 min 4 s from sale start	3,414 / 3,416 (log)

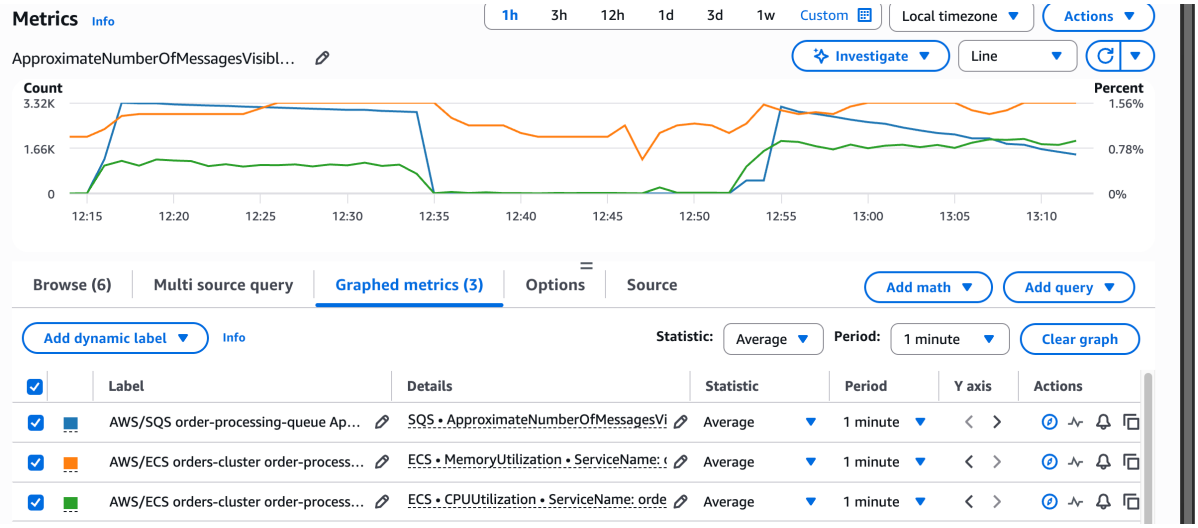
- **Measured rates:** Santosh from the change in processed over steady windows (5 workers 3416 in 2070 s, 20 workers 3360 in 510 s, 100 workers 2990 in 90 s); Sameep from avg_rate and processed per 10 s; Nikhil from NumberOfMessagesDeleted per minute and processed per 15 s.
- **Each 4–5× increase in workers cut drain time by about the same factor** for all three of us.
- **Theory matched measurement (Nikhil's model):** peak ≈ 60 s × (λ − μ) and time to empty ≈ total orders ÷ μ. Nikhil's drains were within ~4% of prediction for 5 and 20 workers (2,072 vs 2,049 s; 530 vs 512 s) and ~20 s late for 100 workers (124 vs 102 s), which fits SQS long-poll round trips and count lag.
- **Processed slightly above accepted** (Santosh +2 at 5 workers, +1 at 20; Nikhil +2 at 100) fits SQS's at-least-once delivery (a rare duplicate) or requests Locust cut off at stop after the receiver had already published them.



Nikhil · Fig 9c. Measured SQS backlog (visible + in flight, every 5 s) for 1, 5, 20 and 100 workers. 1 worker purged at ~5 min; the others drained in 34.5, 8.8 and 2.1 min



Santosh · Fig 9a. 5-worker run alone: queue jumps to ~3.31K at 02:33 and drains in a near-straight line to 0 by ~03:07



Sameep · Fig 9b. Queue depth, memory and CPU for the 1 and 5 worker runs

```
2026-11-09T07:31:22.786000+00:00 ecs/processor/6df1eb2e42ad4993ad91e4f3e6c0cadf 2026/11/09 07:31:22 workers=5 processed=0 in_flight=0 rate_10s=0.00/s avg_rate=0.00/s
2026-11-09T07:31:32.780000+00:00 ecs/processor/6df1eb2e42ad4993ad91e4f3e6c0cadf 2026/11/09 07:31:32 workers=5 processed=0 in_flight=5 rate_10s=0.00/s avg_rate=0.00/s
2026-11-09T07:31:42.781000+00:00 ecs/processor/6df1eb2e42ad4993ad91e4f3e6c0cadf 2026/11/09 07:31:42 workers=5 processed=15 in_flight=5 rate_10s=1.50/s avg_rate=1.35/s
2026-11-09T07:31:52.780000+00:00 ecs/processor/6df1eb2e42ad4993ad91e4f3e6c0cadf 2026/11/09 07:31:52 workers=5 processed=30 in_flight=5 rate_10s=1.50/s avg_rate=1.42/s
2026-11-09T07:32:02.780000+00:00 ecs/processor/6df1eb2e42ad4993ad91e4f3e6c0cadf 2026/11/09 07:32:02 workers=5 processed=50 in_flight=5 rate_10s=2.00/s avg_rate=1.61/s
2026-11-09T07:32:12.780000+00:00 ecs/processor/6df1eb2e42ad4993ad91e4f3e6c0cadf 2026/11/09 07:32:12 workers=5 processed=65 in_flight=5 rate_10s=1.50/s avg_rate=1.58/s
2026-11-09T07:32:22.780000+00:00 ecs/processor/6df1eb2e42ad4993ad91e4f3e6c0cadf 2026/11/09 07:32:22 workers=5 processed=80 in_flight=5 rate_10s=1.50/s avg_rate=1.57/s
```

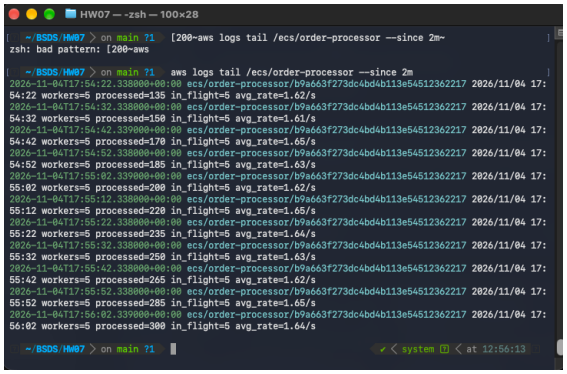
Santosh · Fig 10a. 5 workers: in_flight=5, rate_10s 1.5–2.0/s

```
2026-11-09T07:16:49.159000+00:00 ecs/processor/8572a4dcb19d4be18dfb867c420263e0 2026/11/09 07:16:49 workers=20 processed=0 in_flight=13 rate_10s=0.00/s avg_rate=0.00/s
2026-11-09T07:16:59.160000+00:00 ecs/processor/8572a4dcb19d4be18dfb867c420263e0 2026/11/09 07:16:59 workers=20 processed=60 in_flight=20 rate_10s=6.00/s avg_rate=5.79/s
2026-11-09T07:17:09.160000+00:00 ecs/processor/8572a4dcb19d4be18dfb867c420263e0 2026/11/09 07:17:09 workers=20 processed=120 in_flight=20 rate_10s=6.00/s avg_rate=5.89/s
2026-11-09T07:17:19.160000+00:00 ecs/processor/8572a4dcb19d4be18dfb867c420263e0 2026/11/09 07:17:19 workers=20 processed=190 in_flight=10 rate_10s=7.00/s avg_rate=6.26/s
2026-11-09T07:17:29.159000+00:00 ecs/processor/8572a4dcb19d4be18dfb867c420263e0 2026/11/09 07:17:29 workers=20 processed=260 in_flight=20 rate_10s=7.00/s avg_rate=6.44/s
2026-11-09T07:17:39.159000+00:00 ecs/processor/8572a4dcb19d4be18dfb867c420263e0 2026/11/09 07:17:39 workers=20 processed=320 in_flight=20 rate_10s=6.00/s avg_rate=6.35/s
2026-11-09T07:17:49.159000+00:00 ecs/processor/8572a4dcb19d4be18dfb867c420263e0 2026/11/09 07:17:49 workers=20 processed=380 in_flight=20 rate_10s=6.00/s avg_rate=6.30/s
```

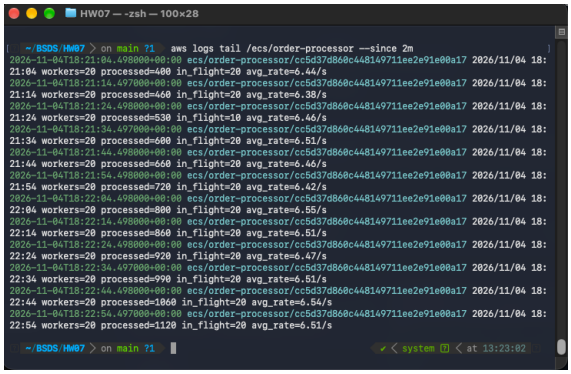
Santosh · Fig 11a. 20 workers: in_flight mostly 20, rate_10s 6–7/s

```
2026-11-09T07:06:18.896000+00:00 ecs/processor/8c57111babb24ab1bb83fc81e291a033 2026/11/09 07:06:18 workers=100 processed=0 in_flight=0 rate_10s=0.00/s avg_rate=0.00/s
2026-11-09T07:06:28.896000+00:00 ecs/processor/8c57111babb24ab1bb83fc81e291a033 2026/11/09 07:06:28 workers=100 processed=0 in_flight=0 rate_10s=0.00/s avg_rate=0.00/s
2026-11-09T07:06:38.896000+00:00 ecs/processor/8c57111babb24ab1bb83fc81e291a033 2026/11/09 07:06:38 workers=100 processed=100 in_flight=100 rate_10s=10.00/s avg_rate=18.99/s
2026-11-09T07:06:48.896000+00:00 ecs/processor/8c57111babb24ab1bb83fc81e291a033 2026/11/09 07:06:48 workers=100 processed=410 in_flight=100 rate_10s=31.00/s avg_rate=26.86/s
2026-11-09T07:06:58.896000+00:00 ecs/processor/8c57111babb24ab1bb83fc81e291a033 2026/11/09 07:06:58 workers=100 processed=735 in_flight=95 rate_10s=32.50/s avg_rate=29.09/s
2026-11-09T07:07:08.896000+00:00 ecs/processor/8c57111babb24ab1bb83fc81e291a033 2026/11/09 07:07:08 workers=100 processed=1092 in_flight=94 rate_10s=35.70/s avg_rate=30.97/s
2026-11-09T07:07:18.896000+00:00 ecs/processor/8c57111babb24ab1bb83fc81e291a033 2026/11/09 07:07:18 workers=100 processed=1400 in_flight=100 rate_10s=30.80/s avg_rate=30.93/s
2026-11-09T07:07:28.896000+00:00 ecs/processor/8c57111babb24ab1bb83fc81e291a033 2026/11/09 07:07:28 workers=100 processed=1720 in_flight=100 rate_10s=32.00/s avg_rate=31.12/s
2026-11-09T07:07:38.896000+00:00 ecs/processor/8c57111babb24ab1bb83fc81e291a033 2026/11/09 07:07:38 workers=100 processed=2066 in_flight=100 rate_10s=34.60/s avg_rate=31.65/s
2026-11-09T07:07:48.896000+00:00 ecs/processor/8c57111babb24ab1bb83fc81e291a033 2026/11/09 07:07:48 workers=100 processed=2400 in_flight=100 rate_10s=33.40/s avg_rate=31.89/s
2026-11-09T07:07:58.896000+00:00 ecs/processor/8c57111babb24ab1bb83fc81e291a033 2026/11/09 07:07:58 workers=100 processed=2700 in_flight=100 rate_10s=30.00/s avg_rate=31.67/s
2026-11-09T07:08:08.896000+00:00 ecs/processor/8c57111babb24ab1bb83fc81e291a033 2026/11/09 07:08:08 workers=100 processed=3043 in_flight=97 rate_10s=34.30/s avg_rate=31.94/s
2026-11-09T07:08:18.896000+00:00 ecs/processor/8c57111babb24ab1bb83fc81e291a033 2026/11/09 07:08:18 workers=100 processed=3400 in_flight=31 rate_10s=35.70/s avg_rate=32.30/s
2026-11-09T07:08:28.901000+00:00 ecs/processor/8c57111babb24ab1bb83fc81e291a033 2026/11/09 07:08:28 workers=100 processed=3431 in_flight=0 rate_10s=3.10/s avg_rate=29.76/s
2026-11-09T07:08:38.896000+00:00 ecs/processor/8c57111babb24ab1bb83fc81e291a033 2026/11/09 07:08:38 workers=100 processed=3431 in_flight=0 rate_10s=0.00/s avg_rate=27.39/s
```

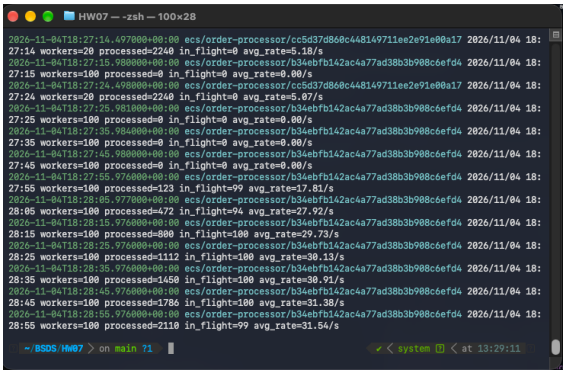
Santosh · Fig 12a. 100 workers: in_flight 94–100, rate_10s 30–35.7/s, ending at processed=3431



Sameep · Fig 10b. 5 workers: avg_rate 1.62–1.65/s



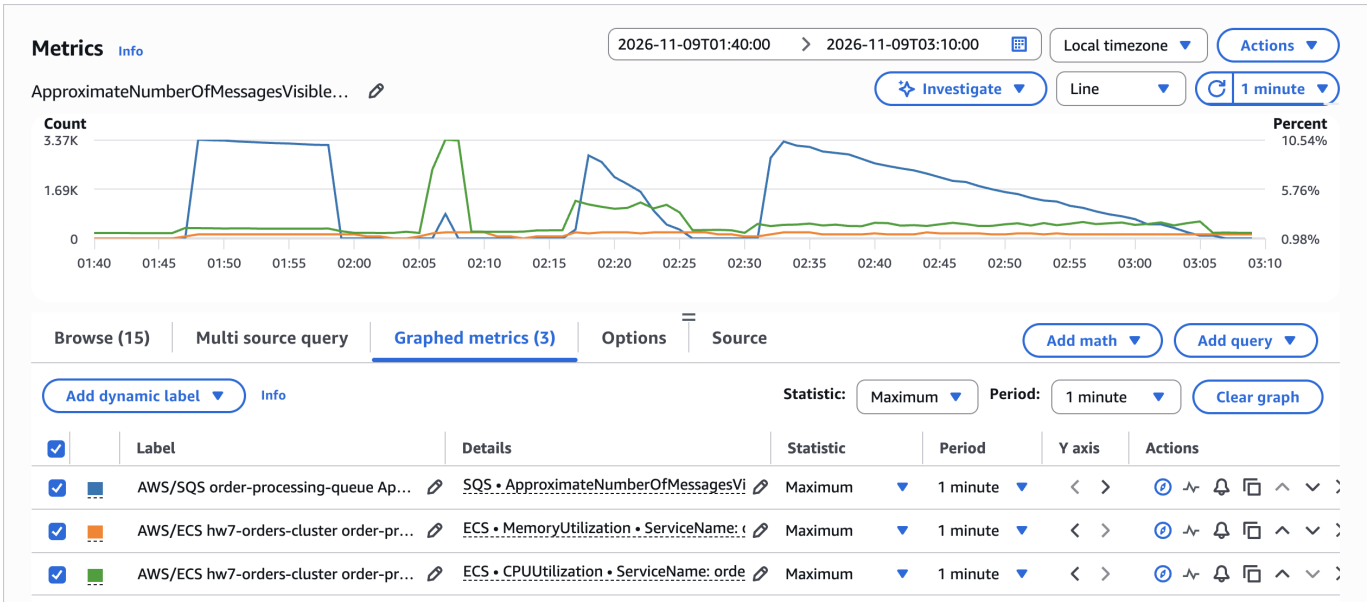
Sameep · Fig 11b. 20 workers: avg_rate ~6.4–6.5/s



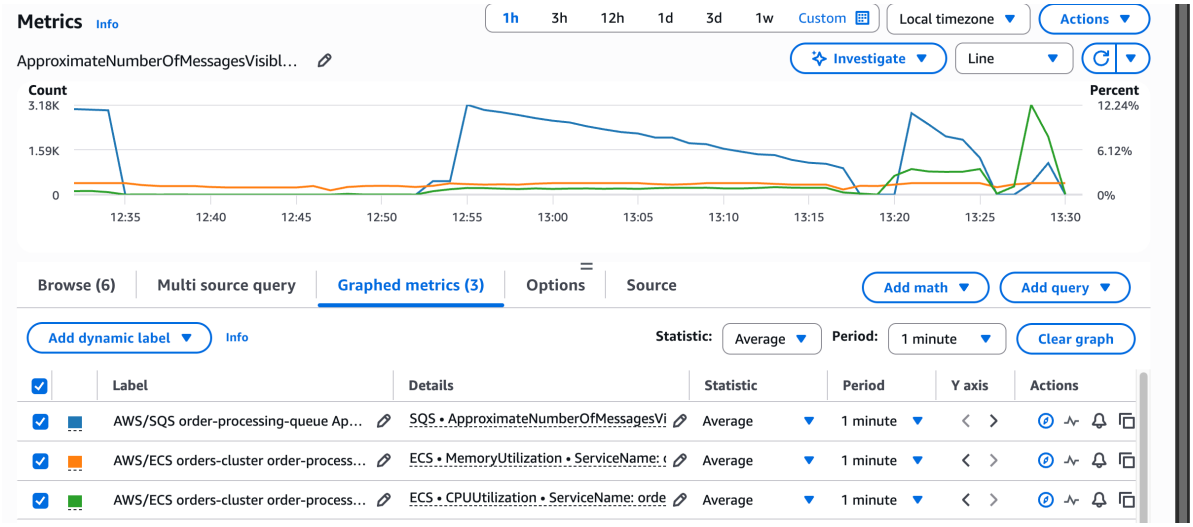
Sameep · Fig 12b. 100 workers: in_flight 99–100, avg_rate up to 31.54/s

```
2026/11/08 04:02:41 stats processed=0 active=100 workers=100
2026/11/08 04:02:56 stats processed=500 active=100 workers=100
2026/11/08 04:03:11 stats processed=1000 active=100 workers=100
2026/11/08 04:03:26 stats processed=1500 active=100 workers=100
2026/11/08 04:03:41 stats processed=2000 active=100 workers=100
2026/11/08 04:03:56 stats processed=2500 active=100 workers=100
2026/11/08 04:04:11 stats processed=3000 active=100 workers=100
2026/11/08 04:04:26 stats processed=3416 active=0 workers=100
```

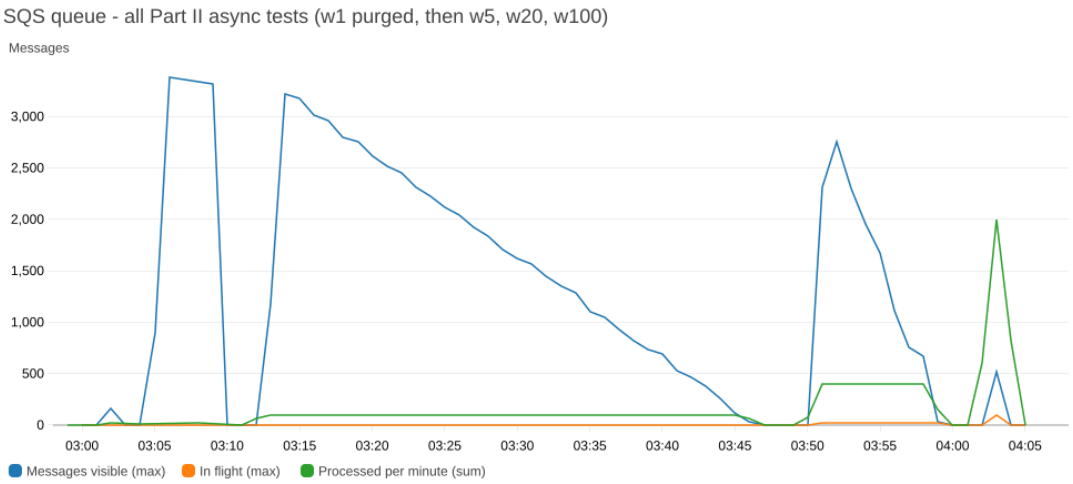
Nikhil · Fig 12c. Processor log with 100 workers: 500 orders per 15 s = 33.3/s, all 100 goroutines busy until empty



Santosh · Fig 13a. Queue depth, CPU and memory across all four runs (01:40–03:10 local)



Sameep · Fig 13b. Queue depth, memory and CPU across the 5, 20 and 100 worker runs



Nikhil · Fig 13c. Messages visible, in flight and processed per minute across all async runs; processed per minute steps up from ~100 to ~400 to ~2,000

5 workers

Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
POST	/orders/async	3420	0	45.62	30	218	65	57.15	0
	Aggregated	3420	0	45.62	30	218	65	57.15	0

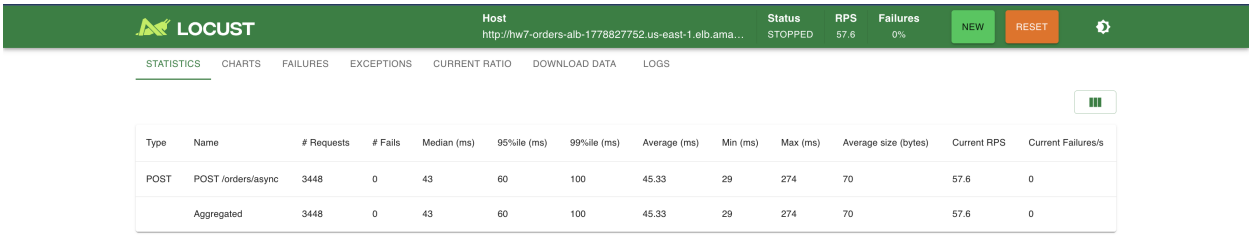
20 workers

Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
POST	/orders/async	3412	0	45.68	30	213	65	57.01	0
	Aggregated	3412	0	45.68	30	213	65	57.01	0

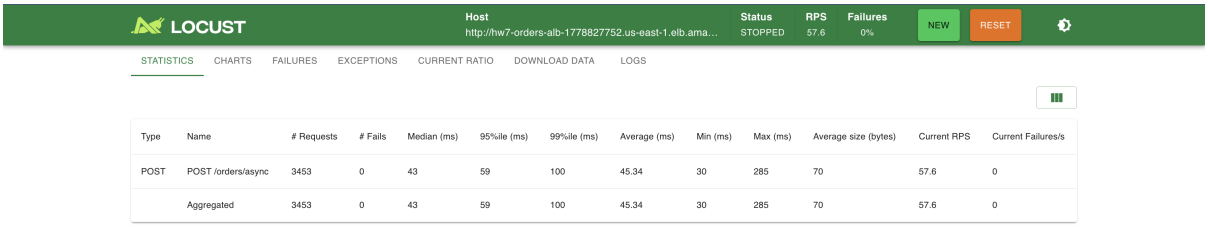
100 workers

Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s
POST	/orders/async	3414	0	45.26	30	220	65	57.04	0
	Aggregated	3414	0	45.26	30	220	65	57.04	0

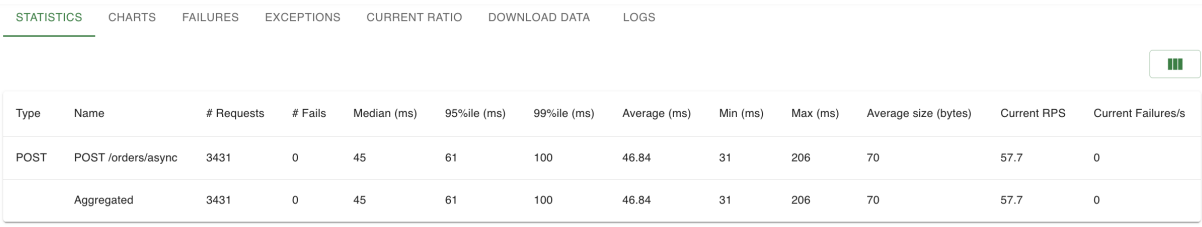
Nikhil · Fig 13d. Locust stats for the 5, 20 and 100 worker flash sales: ~57 RPS and 0 failures each



Santosh · Locust stats, 5 workers: 3448 requests, 0 failures



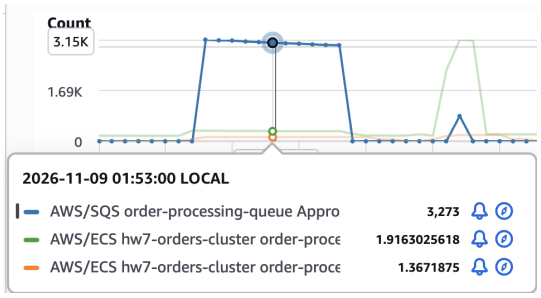
Santosh · Locust stats, 20 workers: 3453 requests, 0 failures



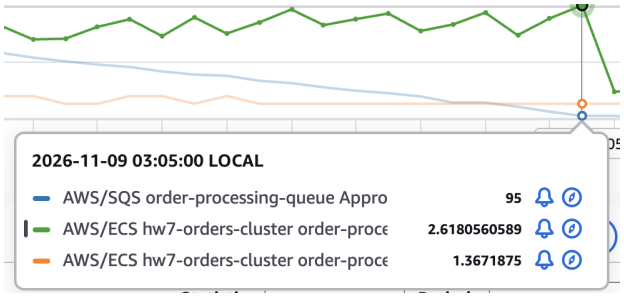
Santosh · Locust stats, 100 workers: 3431 requests, 0 failures

Resource utilization

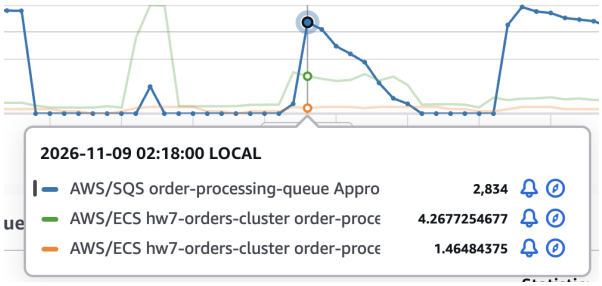
Workers	Santosh: peak CPU / memory	Sameep: CPU / memory	Nikhil: CPU max / avg, memory max
1	1.9163025618% / 1.3671875%	~0.5% / ~1.3–1.5%	0.34% / 0.21%, 1.37%
5	2.6180560589% / 1.3671875%	~0.8% / ~1.3–1.5%	1.55% / 1.08%, 1.56%
20	4.2677254677% / 1.46484375%	~3.5% peak / flat	4.94% / 3.16%, 1.56%
100	10.541211128% / 1.5625%	~12.2% peak / flat	19.98% / 8.21%, 1.56%



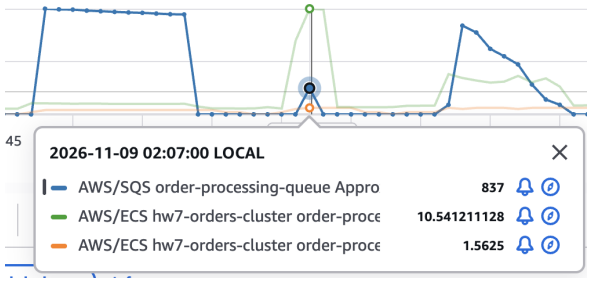
Santosh · 1 worker at 01:53: CPU 1.92%, memory 1.37%



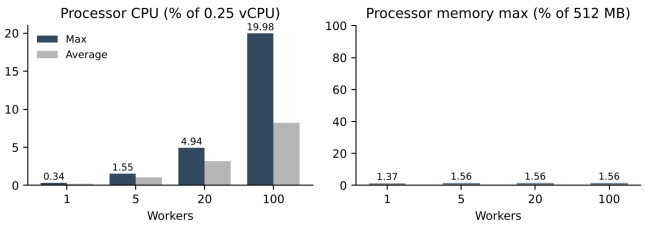
Santosh · 5 workers at 03:05: CPU 2.62%, memory 1.37%



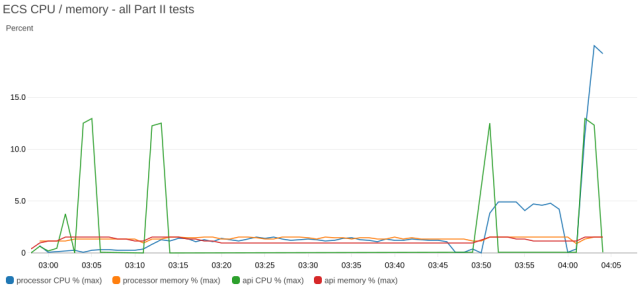
Santosh · 20 workers at 02:18: CPU 4.27%, memory 1.46%



Santosh · 100 workers at 02:07: CPU 10.54%, memory 1.56%



Nikhil · Fig 14c. Processor CPU (max/avg) and memory by worker count



Nikhil · Fig 14d. ECS CPU and memory across all Part II tests

- CPU rises with worker count but stays small for all three of us: 100 goroutines use ~10–20% of a quarter vCPU, because workers spend almost all their time in time.Sleep, like a real payment API call waiting on the network.
- Memory stays at ~1.3–1.6% of 512 MB. A goroutine costs only a few KB.
- The bottleneck is concurrency, not CPU or memory. One small task could run the ~180 workers needed.

Minimum workers

To prevent buildup at 60 orders/s with 3 s per order, Little's Law gives **60 × 3 = 180 workers** (171 at the measured ~57/s). All three data sets agree that 100 is not enough: at 100 workers the queue still grew during the test (peaks 1352 / ~1,250 / 1,507) and only drained after load stopped. With ~20% headroom, ~200–220 workers. In a real system the limit moves downstream: 180 workers means the payment provider must accept 60 calls/s, so production should autoscale on backlog per task, capped at what the provider allows.

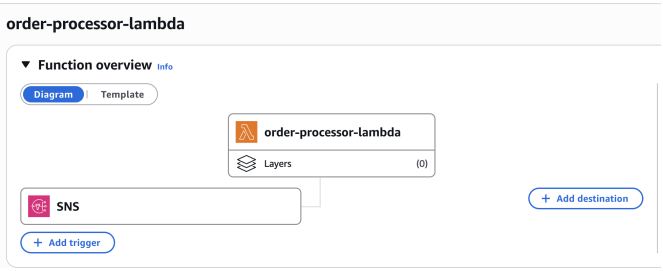
Comparison: measured rates match theory for all three of us, and Santosh's and Nikhil's drain times agree within ~20 s for both 5 and 20 workers when measured from sale start. Santosh and Nikhil both poll only for free workers and got rates at or very near theory at 100 workers (33.22 and 33.3/s); Sameep's fixed pool reached ~31.5/s. Peak depths differ mainly by method: Nikhil counts visible + in flight (so ~100 higher at 100 workers), Santosh counts visible only via the CLI, and Sameep read CloudWatch Average, whose 1-minute points smooth

short spikes. CPU differs in absolute terms (100 workers: 10.54 / ~12.2 / 19.98%) because of the statistic used and differences in logging and polling overhead, but every run stayed far below the task's limits.

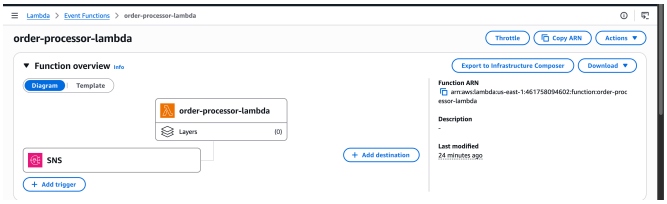
7. Part III: Lambda

Setting	Santosh	Sameep	Nikhil
Function	order-processor-lambda (Go)	order-processor-lambda (Go)	hw07-order-lambda (Go, 2.9 MB)
Runtime	provided.al2, x86_64	provided.al2, x86_64	provided.al2, x86_64
Memory / timeout	512 MB / 10 s	512 MB / 10 s	512 MB / 10 s
Role	LabRole	LabRole	LabRole
Trigger	SNS order-processing-events, direct	SNS order-processing-events, direct	SNS order-processing-events, direct
Safety control	Terraform count toggle, default off	Terraform count toggle, default off	Terraform toggle, default off
Orders sent by curl	9: 1, then 3 one at a time, then 3 at once, then 2 after ~10 min idle	7: 5 about 2 s apart, then 2 after ~10 min idle	7: 5 about 5 s apart, then 2 after ~8 min idle

All three of us enabled the subscription only after every Part II load test had finished and removed it right after. No load testing was run against Lambda. During the curl tests each order also reached SQS and the ECS processor, because SNS delivers to every subscriber.



Santosh · Fig 14a. Lambda console with SNS as the trigger



Sameep · Fig 14b. Lambda console with SNS as the trigger

Cold start observations: Santosh (9 orders)

#	Batch	Log time (UTC)	Instance	Duration	Billed	Init	Type
1	First call	08:21:42	b65d41aa	3004.41 ms	3080 ms	74.68 ms	Cold
2	One at a time, ~5 s apart	08:21:57	b65d41aa	3001.70 ms	3002 ms	—	Warm
3		08:22:03	b65d41aa	3001.70 ms	3002 ms	—	Warm
4		08:22:08	b65d41aa	3002.23 ms	3003 ms	—	Warm
5	3 at once	08:22:15	b65d41aa	3002.11 ms	3003 ms	—	Warm
6		08:22:15	2b7c9fb3	3004.32 ms	3075 ms	70.15 ms	Cold
7		08:22:15	62642826	3002.49 ms	3074 ms	70.64 ms	Cold
8	After ~10 min idle	08:32:28	2b7c9fb3	3002.28 ms	3003 ms	—	Warm
9		08:32:33	2b7c9fb3	3001.88 ms	3002 ms	—	Warm

Cold start observations: Sameep (7 orders)

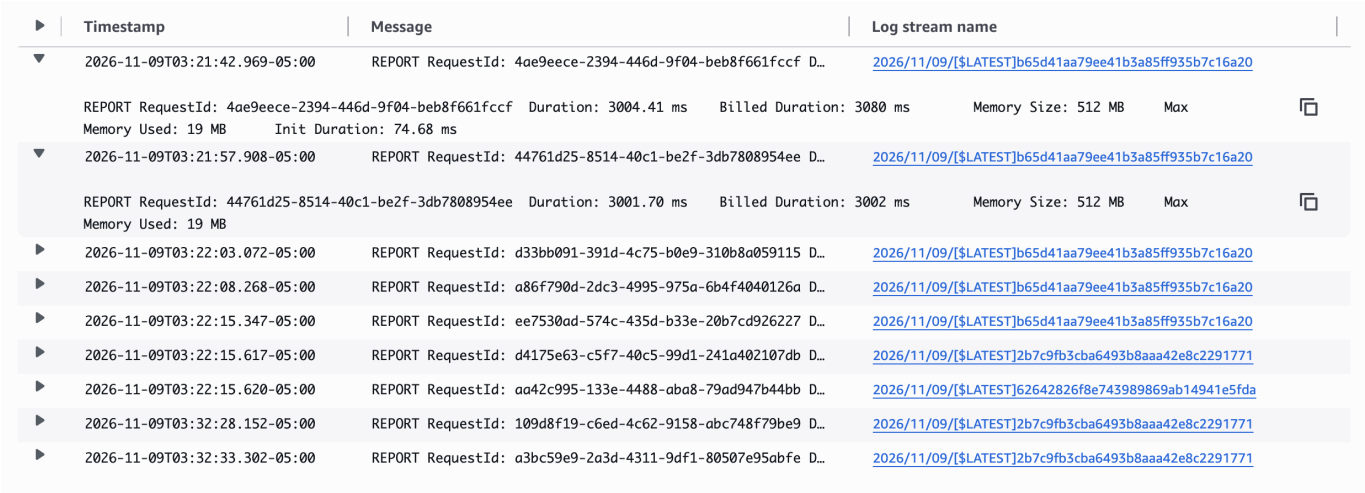
#	When	Duration	Billed	Init	Type
1	First order	3002.77 ms	3075 ms	71.58 ms	Cold

#	When	Duration	Billed	Init	Type
2	Sent 2 s later (instance 1 busy)	3004.64 ms	3094 ms	88.72 ms	Cold
3		3001.74 ms	3002 ms	—	Warm
4		3001.58 ms	3002 ms	—	Warm
5		3001.54 ms	3002 ms	—	Warm
6	After ~10 min idle	3004.28 ms	3075 ms	69.99 ms	Cold
7	5 s after order 6	3001.50 ms	3002 ms	—	Warm

Cold start observations: Nikhil (7 orders)

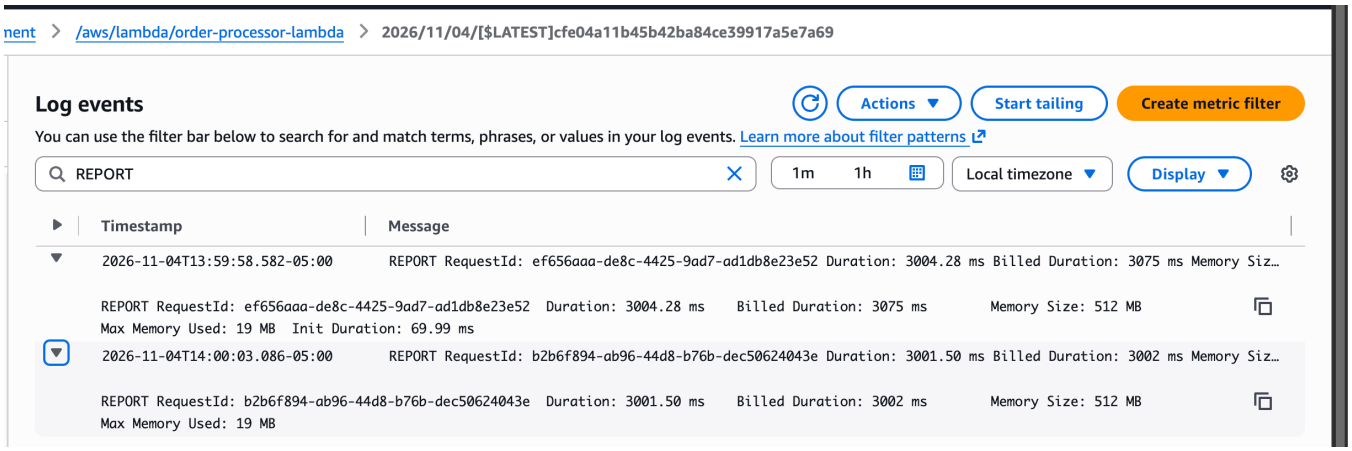
#	Time (UTC)	Instance	Duration	Billed	Init	Type
1	04:05:53	bd8bf430	3002.28 ms	3075 ms	71.90 ms	Cold
2	04:05:58	bd8bf430	3001.54 ms	3002 ms	—	Warm
3	04:06:03	bd8bf430	3003.50 ms	3004 ms	—	Warm
4	04:06:08	bd8bf430	3001.69 ms	3002 ms	—	Warm
5	04:06:13	bd8bf430	3001.52 ms	3002 ms	—	Warm
6	04:14:33 (after ~8 min idle)	fb556e08	3002.58 ms	3075 ms	71.50 ms	Cold
7	04:14:38	fb556e08	3003.72 ms	3004 ms	—	Warm

Max Memory Used was 19 MB in every invocation for all three of us.

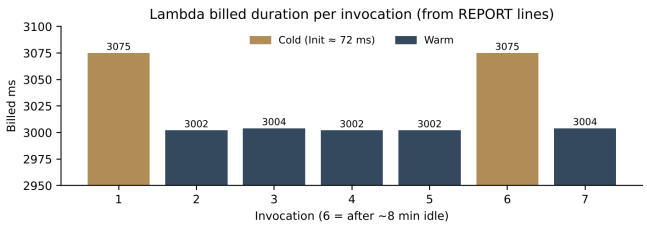


Timestamp	Message	Log stream name
2026-11-09T03:21:42.969-05:00	REPORT RequestId: 4ae9eece-2394-446d-9f04-beb8f661fccf D…	2026/11/09/[$LATEST]b65d41aa79ee41b3a85ff935b7c16a20
	REPORT RequestId: 4ae9eece-2394-446d-9f04-beb8f661fccf Duration: 3004.41 ms Billed Duration: 3080 ms Memory Size: 512 MB Max Memory Used: 19 MB Init Duration: 74.68 ms	
2026-11-09T03:21:57.908-05:00	REPORT RequestId: 44761d25-8514-40c1-be2f-3db7808954ee D…	2026/11/09/[$LATEST]b65d41aa79ee41b3a85ff935b7c16a20
	REPORT RequestId: 44761d25-8514-40c1-be2f-3db7808954ee Duration: 3001.70 ms Billed Duration: 3002 ms Memory Size: 512 MB Max Memory Used: 19 MB	
2026-11-09T03:22:03.072-05:00	REPORT RequestId: d33bb091-391d-4c75-b0e9-310b8a059115 D…	2026/11/09/[$LATEST]b65d41aa79ee41b3a85ff935b7c16a20
2026-11-09T03:22:08.268-05:00	REPORT RequestId: a86f790d-2dc3-4995-975a-6b4f4040126a D…	2026/11/09/[$LATEST]b65d41aa79ee41b3a85ff935b7c16a20
2026-11-09T03:22:15.347-05:00	REPORT RequestId: ee7530ad-574c-435d-b33e-20b7cd926227 D…	2026/11/09/[$LATEST]b65d41aa79ee41b3a85ff935b7c16a20
2026-11-09T03:22:15.617-05:00	REPORT RequestId: d4175e63-c5f7-40c5-99d1-241a402107db D…	2026/11/09/[$LATEST]2b7c9fb3cba6493b8aaa42e8c2291771
2026-11-09T03:22:15.620-05:00	REPORT RequestId: aa42c995-133e-4488-aba8-79ad947b44bb D…	2026/11/09/[$LATEST]62642826f8e743989869ab14941e5fda
2026-11-09T03:32:28.152-05:00	REPORT RequestId: 109d8f19-c6ed-4c62-9158-abc748f79be9 D…	2026/11/09/[$LATEST]2b7c9fb3cba6493b8aaa42e8c2291771
2026-11-09T03:32:33.302-05:00	REPORT RequestId: a3bc59e9-2a3d-4311-9df1-80507e95abfe D…	2026/11/09/[$LATEST]2b7c9fb3cba6493b8aaa42e8c2291771

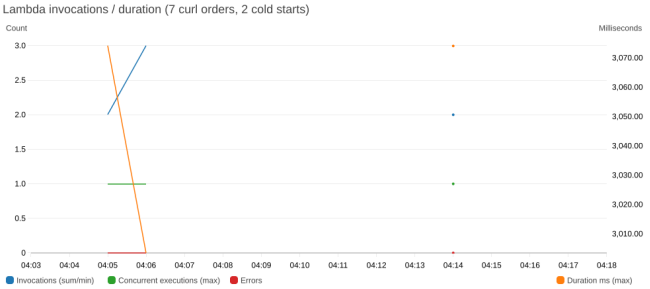
Santosh · Fig 15a. REPORT lines: order 1 cold (Init 74.68 ms, billed 3080 ms) and order 2 warm (billed 3002 ms); log streams show 3 instances



Sameep · Fig 15b. REPORT lines for orders 6 (cold, Init 69.99 ms) and 7 (warm)



Nikhil · Fig 15c. Billed duration for the 7 orders: the two cold starts are billed ~73 ms more



Nikhil · Fig 15d. Lambda invocations, concurrent executions (1), errors (0) and max duration

Findings

Finding	Santosh	Sameep	Nikhil
Cold start frequency	3 of 9	3 of 7	2 of 7
First request	Cold	Cold	Cold
Concurrency	3 at once → 2 new instances (1 → 3, no configuration)	Order 2 arrived while instance 1 was busy	Not triggered: 5 s gaps reused one instance
After idle	Still warm after ~10 min 13 s	Cold after ~10 min	Cold after ~8 min (new instance fb556e08)
Init overhead	70.15–74.68 ms (2.34–2.49%)	69.99–88.72 ms (~2.3–3.0%)	71.50–71.90 ms (≈ 2.4%)
Billing	3074–3080 ms cold vs 3002–3003 ms warm	3075–3094 ms cold vs 3002 ms warm	3075 ms cold vs 3002–3004 ms warm

- **Customer impact (all three):** none. The customer already got HTTP 202 in ~45 ms; a cold start only delays background processing by under 0.1 s. All overheads match the handout's 73 ms / 2.4% example.
- **Memory:** 19 MB used of 512 MB (~27× over-provisioned). 128 MB would cut GB-second cost by 75%.
- **Scale:** SNS invokes one execution per in-flight message, so a ~58/s sale needs ~174 concurrent executions, within the default account limit of 1,000 (Learner Lab may allow less, another reason not to load test it).

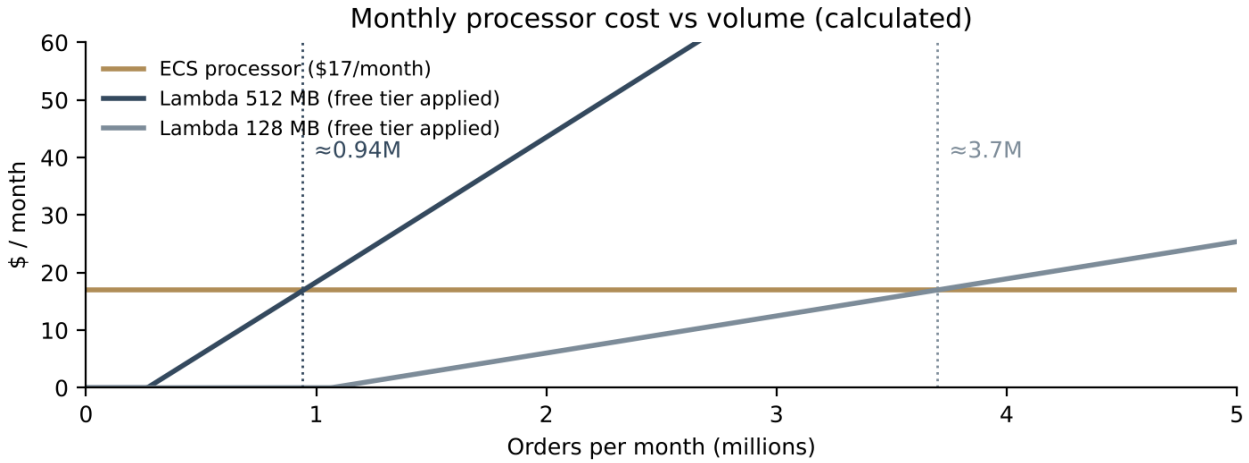
Comparison: all three agree on first-request cold starts and on the ~70–75 ms overhead (Sameep's 88.72 ms is one outlier). Concurrency created new instances whenever orders overlapped (Santosh and Sameep); Nikhil's 5 s gaps never overlapped a 3 s job, so he saw one instance. Idle results differ: cold after ~8 min (Nikhil) and ~10 min (Sameep), but still warm after ~10 min 13 s (Santosh). AWS does not guarantee a fixed idle timeout, so all three results are valid.

8. Cost analysis

All values in this section are calculated, not measured. Pricing (us-east-1, x86): $0.20 per million requests and $0.0000166667 per GB-second. Free tier: 1M requests and 400,000 GB-seconds per month. Per order at 512 MB: 3 s × 0.5 GB = 1.5 GB-seconds.

Scenario	Calculation	Result
Sameep: 10,000 orders/month	15,000 GB-s and 10,000 requests, both in free tier	$0 (vs $17 ECS)
Santosh: 50,000 orders/month	75,000 GB-s and 50,000 requests, both in free tier	$0 (vs $8.50–$17 ECS)
Santosh: 50,000 without free tier	75,000 × 0.0000166667 + 50,000 × 0.0000002	$1.26
Nikhil: ~1.1M orders/month (two 1-min flash sales/week + ~5 orders/s for ~2 busy hours/day)	512 MB: (1.65M − 400K) GB-s × rate + 100K paid requests; 128 MB: (412.5K − 400K) GB-s × rate + requests	512 MB ≈ $20.85; 128 MB ≈ $0.23
Orders covered by free tier	400,000 ÷ 1.5 (512 MB); 400,000 ÷ 0.375 (128 MB)	≈ 267K; ≈ 1.07M
Cost per order beyond free tier	512 MB: 1.5 × 0.0000166667 + 0.0000002	≈ $0.0000252 ($25.22 per million)
One flash sale like ours	≈ 3,400 orders × $0.0000252	≈ $0.09

Break-even	Calculation	Orders/month
vs 1 ECS task ($8.50), 512 MB, free tier	(8.50 ÷ 0.0000166667 + 400,000) ÷ 1.5	≈ 606,667
vs 2 ECS tasks ($17), 512 MB, free tier	(17 ÷ 0.0000166667 + 400,000) ÷ 1.5	≈ 940K–947K
vs $17, 512 MB, no free tier (Nikhil)	17 ÷ 0.0000252	≈ 674,000
vs $17, 128 MB (Sameep, Nikhil)	0.375 GB-s per order; request charges above 1M	≈ 2.6M (no free tier), ≈ 3.7M (free tier)



Nikhil · Fig 16c. Monthly processor cost vs volume with the free tier applied (calculated)

- All three of us get ~940K–947K orders/month for break-even against $17 at 512 MB, not the ~1.7M stated in the assignment. Reaching 1.7M would need about half the per-order compute (~0.75 GB-s, e.g. 256 MB at 3 s).
- The receiver task, ALB (~$16/month) and NAT gateway (~$33/month + data) are needed in both designs. Lambda replaces only the processor, so $8.50 (one task) is the fairer comparison, which brings break-even down to ~607K.
- **Lambda bills for waiting (Nikhil).** The 3 s is an external payment call, and Lambda charges wall-clock time while the function waits. In ECS the same wait happens in goroutines that cost almost

nothing, which is why ECS wins at sustained high volume.

- Our three volumes span the range: 10K and 50K are free; ~1.1M costs slightly more than ECS at 512 MB ($20.85 vs $17) but only ~$0.23 at 128 MB.

9. Trade-offs and recommendation

Gain with Lambda	Loss with Lambda
No queue or worker tuning. In Part II we all changed workers by hand (1, 5, 20, 100), and even 100 could not keep up.	No queue buffer. In Part II, 3,000+ orders waited safely in SQS for over 30 minutes. SNS → Lambda retries a limited number of times, then the order is gone.
Pay per use: $0 at 10K–50K orders/month vs $8.50–$17 for ECS.	Retry control: only SNS delivery retries, and no DLQ unless configured.
Automatic scaling: overlapping orders got new instances with no configuration.	No batching: one SNS record per invocation, where the SQS consumer could receive 10 at a time.
Costs nothing at zero traffic.	Cold starts: ~70–89 ms (2.3–3.0%), and init time is billed.
	Concurrency limits can throttle very large bursts; Lambda bills time spent waiting on the provider.

- **Switch the processor to Lambda and keep the API on ECS.** At low or spiky volume it is free or nearly free, needs no worker scaling, and removes the Phase 4/5 backlog problem.
- **Keep SQS in front of Lambda (SNS → SQS → Lambda).** A flash sale then queues instead of throttling, the 4-day buffer and retries stay, and a DLQ catches failures. Set MaximumConcurrency to the payment provider's limit.
- **Right-size to 128 MB:** the function uses 19 MB, and 128 MB cuts cost by 75%.
- **Cold starts are not a concern** for a 3 s background job the customer is not waiting on.
- **Re-evaluate at sustained volume:** around ~600K orders/month against one ECS task at 512 MB, or ~2–3M orders/month (≈ 1 order/s around the clock) at 128 MB, or if the payment call gets slower, since Lambda cost grows linearly with wait time.

10. Team comparison summary

Measure	Santosh	Sameep	Nikhil
Sync flash: requests / median / max	19 / 30000 / 56164 ms	19 / 30000 / 56163 ms	19 / 30000 / 56086 ms
Async 1 worker: accepted / median / p95	3400 / 42 / 62 ms	3343 / 47 / 110 ms	3414 / 43 / 56 ms
Async vs sync accepted	178.95×	~176×	~180×
1 worker: rate / peak	0.3303/s / 3378	~0.35/s / ~3,320	0.333/s / 3,396
5 workers: rate / peak / time to empty	1.650/s / 3345 / 34 min 52 s*	~1.65/s / ~3,057+ / ~31 min (proj.)	1.67/s / 3,317 / 34 min 32 s*
20 workers: rate / peak / time to empty	6.588/s / 2974 / 8 min 53 s*	~6.5/s / ~2,900 / not measured	6.67/s / 2,993 / 8 min 50 s*
100 workers: rate / peak / time to empty	33.22/s / 1352 / ~1 min 55 s*	~31.5/s / ~1,250 / ~1–2 min	33.3/s / 1,507 / 2 min 4 s*
Peak processor CPU at 100 workers	10.54%	~12.2%	19.98%
Lambda cold starts / overhead	3 of 9 / 70.15–74.68 ms	3 of 7 / 69.99–88.72 ms	2 of 7 / 71.50–71.90 ms
Cold after idle?	No (warm after ~10 min 13 s)	Yes (~10 min)	Yes (~8 min)

Measure	Santosh	Sameep	Nikhil
Expected volume / Lambda cost	50K / $0	10K / $0	~1.1M / $20.85 (512 MB), $0.23 (128 MB)

* measured from sale start.

Likely reasons for the differences:

- **Async p95 (56 / 62 / 110 ms):** network path and time of day for each laptop running Locust; server-side work is the same.
- **Peak depth:** visible + in flight (Nikhil), visible only via the CLI (Santosh), and CloudWatch Average (Sameep), which smooths short spikes.
- **100-worker rate:** polling only for free workers (Santosh, Nikhil) kept all 100 slots busy and reached theory; Sameep's fixed pool reached ~31.5/s.
- **CPU:** Maximum vs Average statistic and differences in logging and polling overhead; all runs stayed far below limits.
- **Lambda idle:** AWS does not guarantee how long an idle instance stays warm.

Shared conclusions: sync is capped at ~0.33/s by the payment slot; async accepts ~176–180× more orders but needs ~180 workers to keep up at 60/s; the processor is concurrency-bound, not CPU-bound; and Lambda is the right processor for our volumes as long as SQS stays in front of it.